Online Shopping and Inventory System

DBMS Mini Project Report

Database: MySQL 8.x | Difficulty: Easy

Concept	Where it is used
Joins	Order details, customer spending, low-stock and payment reports
Triggers	Stock check, stock deduction with low-stock alert, automatic Paid status
Stored procedures	place_order, add_item_to_order, cancel_order, make_payment, restock_product
Transactions	All-or-nothing order placement and cancellation with rollback

Contents

1. Abstract 3

2. Introduction 3

2.1 Background . . . 3

2.2 Problem Statement . . . 3

2.3 Existing System vs Proposed System . . . 3

2.4 Technologies Used . . . 3

3. Objectives 4

4. Scope and Requirements 4

5. Database Design 4

5.1 Tables . . . 4

5.2 Table Structure . . . 5

5.3 Relationships . . . 5

5.4 Entity-Relationship Diagram . . . 6

5.5 Integrity Constraints . . . 6

5.6 Normalization . . . 6

5.7 Data Dictionary . . . 7

6. Triggers 8

7. Stored Procedures and Transactions 8

8. Join Queries 9

9. Implementation: Complete SQL Code 9

9.1 Database and Table Creation . . . 9

9.2 Sample Data . . . 10

9.3 Triggers . . . 11

9.4 Stored Procedures . . . 11

9.5 Test Script . . . 13

9.6 Report Queries . . . 13

10. Testing and Results 14

10.1 Sample Data Loaded . . . 14

10.2 Query Output After the Test Script . . . 15

11. Limitations 16

12. Future Scope 16

13. Conclusion 16

Appendix 16

1. Abstract

This project implements a relational database for an online store. It manages customers, products, suppliers, orders, order items, payments and stock levels. Business rules are enforced inside the database through constraints, triggers and stored procedures, and multi-step operations such as placing an order are wrapped in transactions so they either complete fully or not at all. The project demonstrates joins, triggers, stored procedures and transactions on a normalized schema.

2. Introduction

Online shopping lets customers buy at any hour, but every sale touches stock, payment and supplier records at the same moment. When these live in separate spreadsheets, stores oversell items, lose track of payments and reorder too late. A database management system keeps all of this in one consistent place and enforces the rules automatically.

2.1 Background

A small online store needs to know who its customers are, what it sells, how much stock is left, which supplier provides each product, and whether an order has been paid. These facts are connected: a single order changes the order list, the stock count and the payment record. A relational database stores each kind of fact once, links them with keys, and lets SQL answer questions across all of them.

2.2 Problem Statement

A manual or spreadsheet-based store runs into the following problems:

- Stock counts are updated by hand, so the same item can be sold twice.
- Payments are recorded separately from orders, so unpaid orders go unnoticed.
- Nobody is warned when an item is running low, so reordering is late.
- A failed or partly completed order leaves the records inconsistent.
- Sales and customer reports take hours to prepare.

2.3 Existing System vs Proposed System

Aspect	Manual or spreadsheet system	Proposed database system
Stock accuracy	Updated by hand, often outdated	Deducted automatically by a trigger
Overselling	Possible	Blocked by a stock-check trigger
Payment tracking	Kept in a separate file	Linked to the order; status updates automatically
Data consistency	Partial updates can remain	Transactions commit fully or roll back
Reorder warnings	None	Stock_Alerts rows created at the reorder level
Reporting	Manual filtering and copying	SQL joins give instant reports

2.4 Technologies Used

Component	Choice	Role
Database server	MySQL 8.x	Stores data and runs triggers and procedures

Component	Choice	Role
Language	SQL (DDL, DML and stored-program SQL)	Defines tables, manipulates data, implements business rules
Client tool	MySQL Workbench or the mysql command line	Runs the script and the queries

3. Objectives

- Design a normalized database for online shopping and inventory management.
- Prevent overselling by checking stock automatically before an order line is accepted.
- Keep inventory accurate by deducting stock when an item is ordered and restoring it when an order is cancelled.
- Warn when stock falls to a reorder level so the right supplier can be contacted.
- Guarantee data consistency using transactions with rollback on failure.
- Produce useful business reports using joins.

4. Scope and Requirements

Functional requirements

- Store customer, product and supplier details.
- Let a customer place an order containing one or more products.
- Record payments against an order and update the order status automatically.
- Cancel an order and return the stock.
- Restock products from suppliers.
- Report on sales, customer spending, unpaid balances and low stock.

Software requirements

MySQL Server 8.x and MySQL Workbench or the mysql command-line client.

5. Database Design

5.1 Tables

The system has seven tables. The five core tables are Customers, Products, Orders, Order_Items and Payments. Suppliers and Stock_Alerts were added so that suppliers and low-stock warnings could be handled properly.

Table	Purpose	Primary key (starts at)
Suppliers	Vendors who supply products	supplier_id (101)
Customers	Registered shoppers	customer_id (1001)
Products	Catalogue, price and stock levels	product_id (5001)
Orders	One row per customer order	order_id (10001)
Order_Items	Products contained in each order	item_id (50001)
Payments	Payments made against orders	payment_id (70001)

Table	Purpose	Primary key (starts at)
Stock_Alerts	Low-stock log filled by a trigger	alert_id (90001)

Each table uses AUTO_INCREMENT with a different starting number, so an ID immediately shows which table it belongs to.

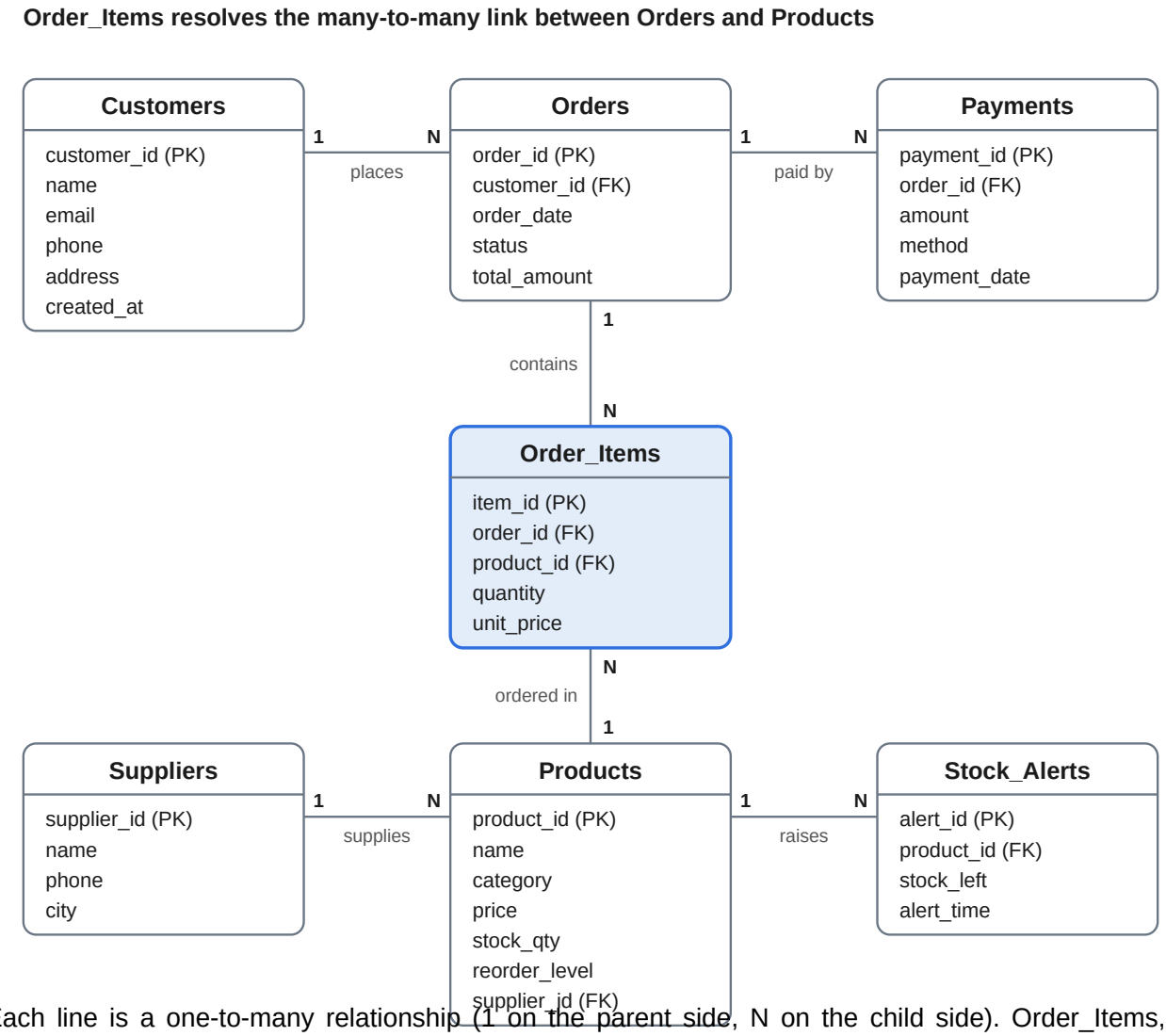
5.2 Table Structure

Table	Columns
Suppliers	supplier_id, name, phone, city
Customers	customer_id, name, email (unique), phone, address, created_at
Products	product_id, name, category, price, stock_qty, reorder_level, supplier_id (FK)
Orders	order_id, customer_id (FK), order_date, status, total_amount
Order_Items	item_id, order_id (FK), product_id (FK), quantity, unit_price
Payments	payment_id, order_id (FK), amount, method, payment_date
Stock_Alerts	alert_id, product_id (FK), stock_left, alert_time

5.3 Relationships

- One Supplier supplies many Products (1:N).
- One Customer places many Orders (1:N).
- One Order contains many Order_Items, and one Product appears in many Order_Items. Order_Items is the link table that resolves the many-to-many relationship between Orders and Products.
- One Order can have many Payments (1:N), which allows partial or split payments.
- One Product can generate many Stock_Alerts (1:N).

5.4 Entity-Relationship Diagram



Each line is a one-to-many relationship (1 on the parent side, N on the child side). Order_Items, highlighted, sits between Orders and Products so one order can hold many products and one product can appear in many orders.

5.5 Integrity Constraints

- Primary and foreign keys on every table.
- UNIQUE on customer email.
- CHECK constraints: price must be greater than 0, stock_qty cannot be negative, quantity must be greater than 0.
- ENUM columns restrict order status (Pending, Paid, Shipped, Delivered, Cancelled) and payment method (Card, UPI, NetBanking, COD).

5.6 Normalization

The schema is in Third Normal Form. Every column depends only on its table's key: customer details are stored once in Customers, supplier details once in Suppliers, and orders reference them by ID rather than repeating them. The unit_price in Order_Items is a deliberate copy of the product price at the time of sale, so later price changes do not alter the history of old orders.

5.7 Data Dictionary

Suppliers

Column	Data type	Constraints	Description
supplier_id	INT	Primary key, AUTO_INCREMENT from 101	Unique supplier number
name	VARCHAR(100)	NOT NULL	Supplier company name
phone	VARCHAR(15)		Contact number
city	VARCHAR(50)		City of the supplier

Customers

Column	Data type	Constraints	Description
customer_id	INT	Primary key, AUTO_INCREMENT from 1001	Unique customer number
name	VARCHAR(100)	NOT NULL	Customer full name
email	VARCHAR(100)	UNIQUE, NOT NULL	Login and contact email
phone	VARCHAR(15)		Contact number
address	VARCHAR(200)		Delivery address
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Registration time

Products

Column	Data type	Constraints	Description
product_id	INT	Primary key, AUTO_INCREMENT from 5001	Unique product number
name	VARCHAR(100)	NOT NULL	Product name
category	VARCHAR(50)		Electronics, Home, Books and so on
price	DECIMAL(10,2)	NOT NULL, CHECK price > 0	Current selling price
stock_qty	INT	NOT NULL, DEFAULT 0, CHECK stock_qty >= 0	Units available
reorder_level	INT	NOT NULL, DEFAULT 5	Stock level that triggers an alert
supplier_id	INT	Foreign key to Suppliers	Who supplies the product

Orders

Column	Data type	Constraints	Description
order_id	INT	Primary key, AUTO_INCREMENT from 10001	Unique order number
customer_id	INT	NOT NULL, foreign key to Customers	Who placed the order
order_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the order was placed
status	ENUM	DEFAULT 'Pending'	Pending, Paid, Shipped, Delivered or Cancelled
total_amount	DECIMAL(12,2)	DEFAULT 0	Sum of all order lines

Order_Items

Column	Data type	Constraints	Description
item_id	INT	Primary key, AUTO_INCREMENT from 50001	Unique order line number
order_id	INT	NOT NULL, foreign key to Orders	Order the line belongs to
product_id	INT	NOT NULL, foreign key to Products	Product bought
quantity	INT	NOT NULL, CHECK quantity > 0	Units bought
unit_price	DECIMAL(10,2)	NOT NULL	Price at the time of sale

Payments

Column	Data type	Constraints	Description
payment_id	INT	Primary key, AUTO_INCREMENT from 70001	Unique payment number
order_id	INT	NOT NULL, foreign key to Orders	Order being paid
amount	DECIMAL(12,2)	NOT NULL	Amount paid
method	ENUM	NOT NULL	Card, UPI, NetBanking or COD
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the payment was made

Stock_Alerts

Column	Data type	Constraints	Description
alert_id	INT	Primary key, AUTO_INCREMENT from 90001	Unique alert number
product_id	INT	Foreign key to Products	Product running low
stock_left	INT		Units left when the alert fired
alert_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	When the alert was raised

6. Triggers

Trigger	Fires	Purpose
trg_check_stock	BEFORE INSERT on Order_Items	Rejects the order line with an error if requested quantity exceeds available stock
trg_reduce_stock	AFTER INSERT on Order_Items	Deducts the ordered quantity from Products and inserts a Stock_Alerts row when stock is at or below the reorder level
trg_mark_paid	AFTER INSERT on Payments	Sums all payments for the order and sets the order status to Paid once the total is covered

Together these keep inventory and order status correct without any application code.

7. Stored Procedures and Transactions

Procedure	What it does	Uses transaction
place_order	Creates an order, adds the item at the current price and calculates the total	Yes
add_item_to_order	Adds another product to an existing order and recalculates the total	Yes
cancel_order	Returns stock for every item in the order and marks it Cancelled	Yes
make_payment	Records a payment, which fires trg_mark_paid	No
restock_product	Increases a product's stock after a supplier delivery	No

The transactional procedures follow the same pattern: START TRANSACTION, perform the steps, COMMIT. An exit handler for SQL exceptions issues a ROLLBACK, so an error part-way through (for example, the stock trigger rejecting a line) leaves no half-created order behind. This protects the ACID properties, especially atomicity and consistency.

```
CALL place_order(1001, 5001, 2, @oid);
```

8. Join Queries

Report	Join type	Business use
Full order details (customer, product, quantity, total, status)	INNER JOIN across four tables	Order history and invoices
Customers who never ordered	LEFT JOIN with IS NULL	Marketing follow-up
Total spent per customer	LEFT JOIN with GROUP BY	Identifying top customers
Low-stock products with supplier name and phone	INNER JOIN	Restocking decisions
Paid amount and balance due per order	LEFT JOIN with SUM	Tracking unpaid orders
Best-selling products	INNER JOIN with GROUP BY	Sales analysis
Stock alerts with product names	INNER JOIN	Inventory monitoring

9. Implementation: Complete SQL Code

This section contains the full MySQL script in the order it is run: tables, sample data, triggers, stored procedures, a test script and the report queries. The same script is available as shopping_inventory_system.sql.

9.1 Database and Table Creation

```
DROP DATABASE IF EXISTS shop_db;
CREATE DATABASE shop_db;
USE shop_db;

CREATE TABLE Suppliers (
    supplier_id   INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    phone         VARCHAR(15),
    city          VARCHAR(50)
) AUTO_INCREMENT = 101;

CREATE TABLE Customers (
    customer_id   INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE NOT NULL,
    phone         VARCHAR(15),
    address       VARCHAR(200),
```

```
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
) AUTO_INCREMENT = 1001;

CREATE TABLE Products (
    product_id     INT AUTO_INCREMENT PRIMARY KEY,
    name           VARCHAR(100) NOT NULL,
    category       VARCHAR(50),
    price          DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_qty      INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    reorder_level  INT NOT NULL DEFAULT 5,
    supplier_id    INT,
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
) AUTO_INCREMENT = 5001;

CREATE TABLE Orders (
    order_id      INT AUTO_INCREMENT PRIMARY KEY,
    customer_id   INT NOT NULL,
    order_date    DATETIME DEFAULT CURRENT_TIMESTAMP,
    status        ENUM('Pending','Paid','Shipped','Delivered','Cancelled')
                  DEFAULT 'Pending',
    total_amount  DECIMAL(12,2) DEFAULT 0,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
) AUTO_INCREMENT = 10001;

CREATE TABLE Order_Items (
    item_id      INT AUTO_INCREMENT PRIMARY KEY,
    order_id     INT NOT NULL,
    product_id   INT NOT NULL,
    quantity     INT NOT NULL CHECK (quantity > 0),
    unit_price   DECIMAL(10,2) NOT NULL,   -- price frozen at time of order
    FOREIGN KEY (order_id)   REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
) AUTO_INCREMENT = 50001;

CREATE TABLE Payments (
    payment_id    INT AUTO_INCREMENT PRIMARY KEY,
    order_id      INT NOT NULL,
    amount        DECIMAL(12,2) NOT NULL,
    method        ENUM('Card','UPI','NetBanking','COD') NOT NULL,
    payment_date  DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
) AUTO_INCREMENT = 70001;

-- Filled automatically by a trigger when stock runs low
CREATE TABLE Stock_Alerts (
    alert_id     INT AUTO_INCREMENT PRIMARY KEY,
    product_id   INT,
    stock_left   INT,
    alert_time   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
) AUTO_INCREMENT = 90001;
```

9.2 Sample Data

```
INSERT INTO Suppliers (name, phone, city) VALUES
('TechSource Pvt Ltd', '9876500001', 'Delhi'),
('HomeGoods Traders',  '9876500002', 'Ludhiana'),
('BookWorld Distributors','9876500003','Mumbai');

INSERT INTO Customers (name, email, phone, address) VALUES
('Amit Sharma',  'amit@example.com',  '9811100001', 'Model Town, Ludhiana'),
('Priya Singh',  'priya@example.com', '9811100002', 'Sector 17, Chandigarh'),
('Rahul Verma',  'rahul@example.com', '9811100003', 'Civil Lines, Jalandhar'),
('Neha Gupta',   'neha@example.com',  '9811100004', 'Sarabha Nagar, Ludhiana');

INSERT INTO Products (name, category, price, stock_qty, reorder_level, supplier_id) VALUES
('Wireless Mouse',     'Electronics', 599.00,  50, 10, 101),
('USB-C Cable',        'Electronics', 249.00, 100, 20, 101),
('Bluetooth Speaker',  'Electronics',1999.00,  15,  5, 101),
('Steel Water Bottle', 'Home',        399.00,  40, 10, 102),
('Desk Lamp',          'Home',        899.00,   8,  5, 102),
('SQL Made Easy',      'Books',       499.00,  25,  5, 103);
```

9.3 Triggers

```
DELIMITER //

-- 3a. Block an order line if there is not enough stock
CREATE TRIGGER trg_check_stock
BEFORE INSERT ON Order_Items
FOR EACH ROW
BEGIN
    DECLARE available INT;
    SELECT stock_qty INTO available
    FROM Products WHERE product_id = NEW.product_id;

    IF available < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock for this product';
    END IF;
END//

-- 3b. Reduce stock after an order line is added, and raise low-stock alert
CREATE TRIGGER trg_reduce_stock
AFTER INSERT ON Order_Items
FOR EACH ROW
BEGIN
    DECLARE left_qty INT;
    DECLARE reorder INT;

    UPDATE Products
       SET stock_qty = stock_qty - NEW.quantity
     WHERE product_id = NEW.product_id;

    SELECT stock_qty, reorder_level INTO left_qty, reorder
    FROM Products WHERE product_id = NEW.product_id;

    IF left_qty <= reorder THEN
        INSERT INTO Stock_Alerts (product_id, stock_left)
        VALUES (NEW.product_id, left_qty);
    END IF;
END//

-- 3c. Mark an order as Paid when full payment has been received
CREATE TRIGGER trg_mark_paid
AFTER INSERT ON Payments
FOR EACH ROW
BEGIN
    DECLARE paid DECIMAL(12,2);
    DECLARE total DECIMAL(12,2);

    SELECT SUM(amount) INTO paid FROM Payments WHERE order_id = NEW.order_id;
    SELECT total_amount INTO total FROM Orders WHERE order_id = NEW.order_id;

    IF paid >= total THEN
        UPDATE Orders SET status = 'Paid' WHERE order_id = NEW.order_id;
    END IF;
END//
```

9.4 Stored Procedures

```
-- 4a. Place an order with one product. All-or-nothing via transaction.
CREATE PROCEDURE place_order(
    IN  p_customer_id INT,
    IN  p_product_id  INT,
    IN  p_quantity    INT,
    OUT p_order_id    INT
)
BEGIN
    DECLARE v_price DECIMAL(10,2);

    -- If anything fails (e.g. insufficient stock), undo everything
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_order_id = NULL;
        SELECT 'Order FAILED - rolled back' AS result;
    END;

    START TRANSACTION;
```

```
        SELECT price INTO v_price FROM Products WHERE product_id = p_product_id;

        INSERT INTO Orders (customer_id) VALUES (p_customer_id);
        SET p_order_id = LAST_INSERT_ID();

        INSERT INTO Order_Items (order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, p_product_id, p_quantity, v_price);   -- triggers fire here

        UPDATE Orders
           SET total_amount = v_price * p_quantity
         WHERE order_id = p_order_id;

    COMMIT;
    SELECT CONCAT('Order ', p_order_id, ' placed successfully') AS result;
END//

-- 4b. Add another item to an existing pending order
CREATE PROCEDURE add_item_to_order(
    IN p_order_id   INT,
    IN p_product_id INT,
    IN p_quantity   INT
)
BEGIN
    DECLARE v_price DECIMAL(10,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Could not add item - rolled back' AS result;
    END;

    START TRANSACTION;
        SELECT price INTO v_price FROM Products WHERE product_id = p_product_id;

        INSERT INTO Order_Items (order_id, product_id, quantity, unit_price)
        VALUES (p_order_id, p_product_id, p_quantity, v_price);

        UPDATE Orders
           SET total_amount = (SELECT SUM(quantity * unit_price)
                               FROM Order_Items WHERE order_id = p_order_id)
         WHERE order_id = p_order_id;
    COMMIT;
    SELECT 'Item added' AS result;
END//

-- 4c. Cancel an order and put stock back
CREATE PROCEDURE cancel_order(IN p_order_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Cancel failed' AS result;
    END;

    START TRANSACTION;
        UPDATE Products p
          JOIN Order_Items oi ON oi.product_id = p.product_id
           SET p.stock_qty = p.stock_qty + oi.quantity
         WHERE oi.order_id = p_order_id;

        UPDATE Orders SET status = 'Cancelled' WHERE order_id = p_order_id;
    COMMIT;
    SELECT 'Order cancelled and stock restored' AS result;
END//

-- 4d. Record a payment
CREATE PROCEDURE make_payment(
    IN p_order_id INT,
    IN p_amount   DECIMAL(12,2),
    IN p_method   VARCHAR(20)
)
BEGIN
    INSERT INTO Payments (order_id, amount, method)
    VALUES (p_order_id, p_amount, p_method);   -- trg_mark_paid fires
    SELECT 'Payment recorded' AS result;
END//

-- 4e. Restock a product from its supplier
```

```
CREATE PROCEDURE restock_product(IN p_product_id INT, IN p_qty INT)
BEGIN
    UPDATE Products SET stock_qty = stock_qty + p_qty
     WHERE product_id = p_product_id;
    SELECT 'Stock updated' AS result;
END//

DELIMITER ;
```

9.5 Test Script

```
-- IDs start at: Suppliers 101, Customers 1001, Products 5001, Orders 10001,
-- Order_Items 50001, Payments 70001, Stock_Alerts 90001

-- Successful order: Amit (1001) buys 2 Wireless Mice
CALL place_order(1001, 5001, 2, @oid);
SELECT @oid AS new_order_id;

-- Add a second item to the same order
CALL add_item_to_order(@oid, 5004, 1);

-- Pay for it (2*599 + 399 = 1597)
CALL make_payment(@oid, 1597.00, 'UPI');

-- Failed order: only 8 Desk Lamps in stock -> trigger blocks, transaction rolls back
CALL place_order(1002, 5005, 20, @oid2);

-- Order that drops stock below reorder level -> creates a Stock_Alert
CALL place_order(1003, 5005, 4, @oid3);

-- Cancel that order and restore stock
CALL cancel_order(@oid3);
```

9.6 Report Queries

```
-- 6a. Full order details (INNER JOINs across 4 tables)
SELECT o.order_id, c.name AS customer, p.name AS product,
       oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_total, o.status
FROM Orders o
JOIN Customers   c  ON c.customer_id = o.customer_id
JOIN Order_Items oi ON oi.order_id   = o.order_id
JOIN Products    p  ON p.product_id  = oi.product_id
ORDER BY o.order_id;

-- 6b. Customers who have never ordered (LEFT JOIN)
SELECT c.customer_id, c.name
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- 6c. Total spent per customer (excluding cancelled orders)
SELECT c.name, COUNT(o.order_id) AS orders, COALESCE(SUM(o.total_amount),0) AS total_spent
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;

-- 6d. Products at or below reorder level, with supplier to contact
SELECT p.name AS product, p.stock_qty, p.reorder_level,
       s.name AS supplier, s.phone
FROM Products p
JOIN Suppliers s ON s.supplier_id = p.supplier_id
WHERE p.stock_qty <= p.reorder_level;

-- 6e. Orders with payment status (LEFT JOIN so unpaid orders still show)
SELECT o.order_id, c.name AS customer, o.total_amount,
       COALESCE(SUM(pay.amount),0) AS paid,
       o.total_amount - COALESCE(SUM(pay.amount),0) AS balance_due
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
LEFT JOIN Payments pay ON pay.order_id = o.order_id
GROUP BY o.order_id, c.name, o.total_amount;

-- 6f. Best-selling products
SELECT p.name, SUM(oi.quantity) AS units_sold
```

```
FROM Products p
JOIN Order_Items oi ON oi.product_id = p.product_id
JOIN Orders o       ON o.order_id = oi.order_id AND o.status <> 'Cancelled'
GROUP BY p.product_id, p.name
ORDER BY units_sold DESC;

-- 6g. Stock alerts raised by the trigger
SELECT a.alert_id, p.name, a.stock_left, a.alert_time
FROM Stock_Alerts a
JOIN Products p ON p.product_id = a.product_id;
```

10. Testing and Results

The script includes a test run that exercises each feature.

Test	Action	Expected result
Successful order	Customer 1001 buys 2 Wireless Mice (5001)	Order created, stock drops from 50 to 48, total 1,198
Add item	Add 1 Steel Water Bottle (5004) to that order	Total becomes 1,597, bottle stock drops from 40 to 39
Payment	Pay 1,597 by UPI	trg_mark_paid sets the order status to Paid
Insufficient stock	Customer 1002 orders 20 Desk Lamps (5005) with only 8 in stock	trg_check_stock raises an error, the transaction rolls back, no order or stock change is saved
Low-stock alert	Customer 1003 orders 4 Desk Lamps	Stock falls to 4, which is below the reorder level of 5, so a Stock_Alerts row is created
Cancellation	Cancel the lamp order	Stock returns to 8 and the order status becomes Cancelled

Note: MySQL does not reuse an auto-increment value after a rolled-back insert, so the failed order leaves a gap in the Order IDs. This is normal behaviour.

10.1 Sample Data Loaded

Suppliers

supplier_id	name	phone	city
101	TechSource Pvt Ltd	9876500001	Delhi
102	HomeGoods Traders	9876500002	Ludhiana
103	BookWorld Distributors	9876500003	Mumbai

Customers

customer_id	name	email	address
1001	Amit Sharma	amit@example.com	Model Town, Ludhiana
1002	Priya Singh	priya@example.com	Sector 17, Chandigarh
1003	Rahul Verma	rahul@example.com	Civil Lines, Jalandhar
1004	Neha Gupta	neha@example.com	Sarabha Nagar, Ludhiana

Products

product_id	name	category	price	stock_qty	reorder_level	supplier_id
5001	Wireless Mouse	Electronics	599.00	50	10	101

product_id	name	category	price	stock_qty	reorder_level	supplier_id
5002	USB-C Cable	Electronics	249.00	100	20	101
5003	Bluetooth Speaker	Electronics	1999.00	15	5	101
5004	Steel Water Bottle	Home	399.00	40	10	102
5005	Desk Lamp	Home	899.00	8	5	102
5006	SQL Made Easy	Books	499.00	25	5	103

10.2 Query Output After the Test Script

After the test script in section 9.5 runs, the stock levels are: Wireless Mouse 48, Steel Water Bottle 39, Desk Lamp 8 (restored by the cancellation), and every other product unchanged.

Full order details

order_id	customer	product	quantity	unit_price	line_total	status
10001	Amit Sharma	Wireless Mouse	2	599.00	1198.00	Paid
10001	Amit Sharma	Steel Water Bottle	1	399.00	399.00	Paid
10003	Rahul Verma	Desk Lamp	4	899.00	3596.00	Cancelled

The failed order for customer 1002 left no row at all, because the transaction rolled back. Its order number 10002 was used up and skipped.

Customers who never ordered

customer_id	name
1002	Priya Singh
1004	Neha Gupta

Total spent per customer (cancelled orders excluded)

name	orders	total_spent
Amit Sharma	1	1597.00
Priya Singh	0	0.00
Rahul Verma	0	0.00
Neha Gupta	0	0.00

Payment status per order

order_id	customer	total_amount	paid	balance_due
10001	Amit Sharma	1597.00	1597.00	0.00
10003	Rahul Verma	3596.00	0.00	3596.00

Order 10003 shows a balance because it was cancelled before any payment; cancelled orders should be ignored in this report.

Best-selling products (cancelled orders excluded)

name	units_sold
Wireless Mouse	2
Steel Water Bottle	1

Low-stock query and alert log

The low-stock query returns no rows at the end of the test, because the cancellation restored Desk Lamp stock to 8, above its reorder level of 5. The alert raised while stock stood at 4 stays in the log:

alert_id	name	stock_left
90001	Desk Lamp	4

11. Limitations

- place_order handles one product per call; further products must be added with add_item_to_order.
- cancel_order restores stock but does not reverse or refund payments already recorded.
- There is no user login, shopping cart or delivery tracking.
- Orders are not tied to a specific warehouse or stock batch.

12. Future Scope

- Add Categories, Reviews, Cart and Addresses tables.
- Support multi-item orders in a single procedure call.
- Add refunds and a returns workflow.
- Create views for monthly sales and inventory reports.
- Add indexes on frequently searched columns such as email, product name and order date.
- Connect the database to a web or mobile front end.

13. Conclusion

The project delivers a working online shopping and inventory database that keeps stock, orders and payments consistent. Triggers automate stock control and payment status, stored procedures package business operations, transactions guarantee all-or-nothing behaviour, and join queries provide the reports a store owner needs. The design is normalized, enforces integrity through constraints, and can be extended with additional features without restructuring the core tables.

Appendix

The complete SQL script (table creation, sample data, triggers, procedures, test calls and queries) is provided in the file shopping_inventory_system.sql.